

第二次作业

作业发布时间：2025/05/08 星期四

本次作业要求如下：

1.截止日期：2025/05/19 周一晚 24:00。

2.提交作业给本课程 QQ 群里的助教。

3.命名格式：附件和邮件命名统一为“第二次作业+学号+姓名”，作业为 PDF 格式。

4.注意：

(1) 选择、判断和填空只需要写答案，大题要求有详细过程，过程算分。

(2) 答案请用另一种颜色的笔回答，便于批改，否则视为无效答案。

(3) 大题的过程最好在纸上写了拍照，放到 word 里。

5.本次作业遇到问题请联系课程群里的助教。

一. 选择题

1. 位于存储器层次结构中的最顶部的是(A)。

A. 寄存器 B. 主存 C. 磁盘 D. 高速缓存

2. 关于 Intel 的现代 X86-64 CPU，说法正确的是(B)。

A. 属于 RISC B. 属于 CISC C. 属于 MISC D. 属于 AVX

3. 操作系统在管理硬件时使用了几个抽象概念，其中(A)是对处理器、主存和 I/O 设备的抽象表示。

A. 进程 B. 虚拟存储器 C. 文件 D. 虚拟机

4. 以下有关编程语言的叙述中，正确的是(D)。

A. 计算机能直接执行高级语言程序和汇编语言程序

B. 机器语言可以通过汇编过程变成汇编语言

C. 汇编语言比高级语言有更好的可读性

D. 汇编语言和机器语言都与计算机系统结构相关

5. 给定字长的整数 x 和 y 按补码相加，和为 s ，则发生正溢出的情况是(A)

A. $x>0, y>0, s\leq 0$ B. $x>0, y<0, s\leq 0$

C. $x>0, y<0, s\geq 0$ D. $x<0, y<0, s\geq 0$

6. 设机器数字长 8 位（含 1 位符号位），若机器数 DAH 为补码，分别对其进行算术左移一位和算术右移一位，其结果分别为 (A)

A. B4H, EDH

B. B5H, 6DH

C. B4H, 6DH

D. B5H, EDH

7. -1029 的 16 位补码用十六进制表示为(C)。
- A. 8405H
 - B. 0405H
 - C. FBFBH
 - D. 7BFBH
8. 已知两个正浮点数, $N_1 = 2^{j_1} \times S_1$, $N_2 = 2^{j_2} \times S_2$, 当下列 (A) 成立时, $N_1 < N_2$ 。
- A. S_1 和 S_2 均为规格化数, 且 $J_1 < J_2$
 - B. $S_1 < S_2$
 - C. S_1 和 S_2 均为规格化数, 且 $J_1 > J_2$
 - D. $J_1 < J_2$
9. C 程序执行到整数或浮点变量除以 0 可能发生(D)。
- A. 显示除法溢出错直接退出
 - B. 程序不提示任何错误
 - C. 可由用户程序确定处理办法
 - D. 以上都可能
10. 补码加法运算的溢出判别中, 以下说法正确的是(D)
- A. 符号相同的两个数相加必定不会发生溢出
 - B. 符号不同的两个数相加可能发生溢出
 - C. 符号相同的两个数相加必定发生溢出
 - D. 符号不同的两个数相加不可能发生溢出
11. 假定变量 i、f 的数据类型分别是 int、float。已知 $i=12345$, $f=1.2345e3$, 则在一个 32 位机器中执行下列表达式时, 结果为“假”的是(C)。
- A. $i==(int)(float)i$
 - B. $i==(int)(double)i$
 - C. $f==(float)(int)f$
 - D. $f==(float)(double)f$
12. 以下关系表达式, 结果为“真”的是(B)。
- A. $2147483647U > -2147483648$
 - B. $(unsigned) -1 > -2$
 - C. $-1 < 0U$
 - D. $2147483647 < (int) 2147483648U$
13. 假定某数采用 IEEE 754 单精度浮点数格式表示为 00000001H, 则该数的值是 (B)。
- A. NaN (非数)
 - B. $1.0 \times 2^{(-149)}$
 - C. $1.00...01 \times 2^{(-127)}$
 - D. $1.0 \times 2^{(-150)}$

14. C 语言程序如下，下列说法叙述正确的是(B)。

```
#include <stdio.h>
#define DELTA sizeof(int)
int main(){
    int i;
    for (i = 40; i - DELTA >= 0; i -= DELTA)
        printf("%d ",i);
}
```

- A. 程序有编译错误
- B. 程序输出 11 个数：40 36 32 28 24 20 16 12 8 4 0
- C. 程序死循环，不停地输出数值
- D. 以上都不对

15. 若 int 型变量 x 的最高有效字节全变 0，其余各位不变，则对应 C 语言表达式为(A)。

- A. $((\text{unsigned}) x \ll 8) \gg 8$
- B. $((\text{unsigned}) x \gg 8) \ll 8$
- C. $(x \ll 8) \gg 8$
- D. $(x \gg 8) \ll 8$

二. 填空题

1. 64 位系统中 short 数 -2 的机器数二进制表示__1111,1111,1111,1110__。
2. 判断整型变量 n 的位 7 为 1 的 C 语言表达式是__ $n \& (1 \ll 7)$ __。
3. -1024 采用 IEEE 754 单精度浮点数格式按内存地址从低到高表示的结果（十六进制表示，小端模式）是__00 00 80 C4__。
4. C 语言中的 double 类型浮点数用__64__位表示。
5. 64 位系统中，整型变量 x = -7，其在内存从低地址到高地址依次存放的数是 F9 FF FF FF_ (十六进制表示，小端模式)。

三. 判断题

1. (✓)C 浮点常数 IEEE754 编码的缺省舍入规则是四舍五入。
2. (✓)浮点数 IEEE754 标准中，规格化数比非规格化数多。
3. (✓)对 unsigned int x, $(x * x) \geq 0$ 总成立。
4. (✗)CPU 无法判断加法运算的和是否溢出。
5. (✓)C 语言中的有符号数强制转换成无符号数时位模式不会改变。
6. (✓)C 语言中数值从 int 转换成 double 后，数值虽然不会溢出，但是可能是不精确的。
7. (✗)C 语言中从 double 转换成 float 时，值可能溢出，但不可能被舍入。

四. 分析题

1. 请说明 float 类型编码格式，并按步骤计算 -10.1 的各部分内容，写出 -10.1 在内存从低地址到高地址的存储字节内容（小端系统）。

答：

Float 类型编码格式 (IEEE 754 单精度) 使用 32 位 (4 字节) 来存储一个浮点数。这 32 位被划分为三个部分：

- 符号位 (Sign bit, S)：1 位。0 表示正数，1 表示负数。
- 指数位 (Exponent, E)：8 位。采用偏置表示法 (biased representation)。对于单精度，偏置量 (bias) 为 127。实际指数值 = 存储的指数值 - 127。存储的指数值范围为 0 到 255。全 0 (00000000) 和全 1 (11111111) 是特殊值，用于表示非规格化数、零、无穷大和 NaN (Not a Number)。对于规格化数，指数范围是 1 到 254 (实际指数范围是 -126 到 +127)。
- 尾数位 (Mantissa/Fraction, M)：23 位。对于规格化数，尾数部分表示的是小数部分。在二进制科学计数法中，整数部分总是隐含的 "1" (这个 "1" 不会被存储，从而可以节省一位，提高精度)。因此，实际的尾数值是 1.M。对于非规格化数 (当指数位全为 0 时)，隐含的整数部分为 "0"。

-10.1:

符号位 (S): 1 (表示负数)

指数位 (E) :

原始十进制数: 10.1

二进制表示 (绝对值): 1010.00011001100110011001...

规格化二进制: 1.01000011001100110011001... * 2³

实际指数: 3

偏置指数 (E = 实际指数 + 127): 3 + 127 = 130

指数位的二进制表示 (8 位): 10000010

尾数部分 (M) (规格化后小数部分, 23 位, 已进行舍入):

010 0001 1001 1001 1001 1010

完整 32 位表示: 1100 0001 0010 0001 1001 1001 1001 1010

十六进制表示: C1 21 99 9A

小端存储为: 9A 99 21 C1

2. 写出负 short 型整数 x 除以 2 的整数幂 (k) 的商的公式, 当 x = -17231 时, 计算 $x/2^6$ 的值的十六进制表示, 写出结果在内存从低地址到高地址的存储字节内容 (小端系统)。

答: 负 short 型整数 x 除以 2 的整数幂 k 的商的公式可表示为: $x / (2^k) = x \gg k$, 当 x = -17231 时, 计算 $x/2^6$ 的值: $-17231/64 = -269$ 。 -269 的十六进制表示为 0xFE3。在小端系统中, 从低地址到高地址的存储字节为: 0xF3, 0xFE。

3. 向量元素和计算的相关程序如下, 请改写或重写计算函数 vector_sum, 进行速度优化, 并简要说明优化的依据。(如果能自己动手在电脑上测试一下, 优化前后性能提升了多少会有额外加分, 贴上截图, 注明机器型号)

```

/*向量的数据结构定义 */
typedef struct{
    int len;    //向量长度，即元素的个数
    float *data; //向量元素的存储地址
} vec;

/*获取向量长度*/
int vec_length(vec *v){return v->len;}

/* 获取向量中指定下标的元素值，保存在指针参数 val 中*/
int get_vec_element(*vec v, size_t idx, float *val){
    if (idx >= v->len)
        return 0;
    *val = v->data[idx];
    return 1;
}

/*计算向量元素的和*/
void vector_sum(vec *v, float *sum){
    long int i;
    *sum = 0; //初始化为 0
    for (i = 0; i < vec_length(v); i++) {
        float val;
        get_vec_element(v, i, &val); //获取向量 v 中第 i 个元素的值，存入 val 中
        *sum = *sum + val;    //将 val 累加到 sum 中
    }
}

```

答：

原始的 vector_sum 函数存在以下性能问题：

每次循环都调用 get_vec_element 函数，导致额外的函数调用开销

get_vec_element 函数每次都检查索引边界

逐个元素累加不利于编译器进行自动向量化优化

优化依据：

直接访问数据数组，避免函数调用开销

一次性检查边界条件，避免重复检查

使用循环展开技术，减少循环迭代次数和分支预测失败

应用 SIMD 优化思想，处理多个元素

优化后的代码：

```

void vector_sum_optimized(vec *v, float *sum){
    long int i;
    float *data = v->data;
    int len = v->len;
    float sum1 = 0.0f;
    float sum2 = 0.0f;
    float sum3 = 0.0f;
    float sum4 = 0.0f;

    // 使用4路循环展开
    i = 0;
    for(; i + 3 < len; i += 4) {
        sum1 += data[i];
        sum2 += data[i+1];
        sum3 += data[i+2];
        sum4 += data[i+3];
    }

    // 处理剩余元素
    for(; i < len; i++) {
        sum1 += data[i];
    }

    // 合并结果
    *sum = sum1 + sum2 + sum3 + sum4;
}

```

测试平台：

系统：WSL2 Ubuntu-24.04 LTS

处理器：Intel(R) Core(TM) Ultra 9 275HX 24 核 24 线程

测试方式：

```

int main() {
    // 创建测试向量
    const int SIZE = 10000000; // 一千万个元素
    vec v;
    v.len = SIZE;
    v.data = (float*)malloc(SIZE * sizeof(float));

    // 初始化随机数据
    srand(time(NULL));
    for (int i = 0; i < SIZE; i++) {
        v.data[i] = (float)rand() / RAND_MAX;
    }

    float sum1 = 0.0f, sum2 = 0.0f;
    double start, end;

    // 测试原始函数
    start = get_time();
    for (int i = 0; i < 10; i++) {
        vector_sum(&v, &sum1);
    }
    end = get_time();
    double original_time = end - start;

    // 测试优化函数
    start = get_time();
    for (int i = 0; i < 10; i++) {
        vector_sum_optimized(&v, &sum2);
    }
    end = get_time();
    double optimized_time = end - start;

    // 输出结果
    printf("原始和: %f\n", sum1);
    printf("优化和: %f\n", sum2);
    printf("原始版本耗时: %.6f 秒\n", original_time);
    printf("优化版本耗时: %.6f 秒\n", optimized_time);
    printf("性能提升: %.2f 倍\n", original_time / optimized_time);

    // 释放内存
    free(v.data);
    return 0;
}

```

编译执行后的结果：

```
(base) jin_zd@DESKTOP-Q9M21PV:~/test$ gcc -o vector_test test.c -O2
(base) jin_zd@DESKTOP-Q9M21PV:~/test$ ./vector_test
原始和：5000587.000000
优化和：5000662.000000
原始版本耗时：0.043888 秒
优化版本耗时：0.017042 秒
性能提升：2.58 倍
```